# Akka & Actor Model — Complete Lecture & Lab Notes

**Course:** Networked Software for Distributed Systems (NSDS) — Politecnico di Milano | **Reference:** Lectures 1, 2, 3 & Lab 1 Solutions

## 1. Lecture 1 — Actor Model & Akka Fundamentals

### 1.1 Core Philosophy & Actor Properties

- **"The world is parallel"** (Joe Armstrong, Erlang): concurrent applications are much easier to write when modeled after independent real-world entities that interact by exchanging messages.
- **"Do not communicate by sharing memory; instead, share memory by communicating"** (Effective Go): because actors do not rely on shared memory, the **exact same program logic** can run locally on one machine or across a distributed cluster.
- **Properties of Actors** (the universal primitive of concurrent computation):
  - **Stateful:** each actor encapsulates its own private internal state.
  - **Asynchronous:** actors run concurrently and communicate exclusively through asynchronous message passing.
  - **Persistent:** internal state can optionally be saved to persistent storage for recovery.
- **What can an actor do upon receiving a message?**
  1. **Take local decisions:** modify its private internal state or **change its behavior** for future messages.
  2. **Send messages:** send 0, 1, or more messages to the `sender()` or to any other `ActorRef` it knows.
  3. **Start new actors:** spawn child actors inside its context ( `getContext().actorOf(...)` ).

### 1.2 Shared Memory vs. Actor Model (Bank Account Example)

| Concern | Shared-Memory Model (Threads + Locks) | Actor Model (Akka) |
|---|---|---|
| **Concurrency** | Explicitly create threads to manage one or more accounts. | Creating an actor to manage an account automatically makes it run asynchronously with respect to other actors. |
| **State Safety** | Must add `synchronized` blocks, locks, or atomics to prevent negative balance violations. | **No locks needed:** actors can only affect each other through messages. |
| **Atomicity** | Manual critical sections; error-prone under concurrency. | **Messages are processed atomically:** *"as if the entire processing of a message was executed in a `synchronized` method of a class where all methods are `synchronized`"*. |
| **Concurrent Arrivals** | Threads block contending on monitors/locks. | Messages received meanwhile are **queued up in the mailbox** and do not affect the currently running receive handler. |

### 1.3 Akka Primitives & Basic Counter Template (Slides 16–21)

```java
// Actor definition (CounterActor.java)
public class CounterActor extends AbstractActor {
    private int counter;

    public CounterActor() { this.counter = 0; }

    @Override
    public Receive createReceive() {
        return receiveBuilder()
            .match(SimpleMessage.class, this::onMessage)
            .build();
    }

    void onMessage(SimpleMessage message) {
        ++counter; // Safe without locks!
        System.out.println("Counter increased to: " +
counter);
    }

    static Props props() {
        return Props.create(CounterActor.class);
    }
}
```

```java
// Main bootstrapper (Counter.java)
public class Counter {
    private static final int numThreads = 16;
    private static final int numMessages = 1000;

    public static void main(String[] args) throws
Exception {
        final ActorSystem sys =
ActorSystem.create("System");
        final ActorRef counter =
            sys.actorOf(CounterActor.props(), "counter");

        final ExecutorService exec =
            Executors.newFixedThreadPool(numThreads);
        for (int i = 0; i < numMessages; i++) {
            exec.submit(() -> counter.tell(
                new SimpleMessage(),
ActorRef.noSender()));
        }
        System.in.read();
        exec.shutdown();
        sys.terminate();
    }
}
```

| Primitive / Method | Role & Usage Rules |
|---|---|
| `AbstractActor` | Base class for actors. Must implement `createReceive()`. |
| `Props.create(MyActor.class, args...)` | Property object defining the actor class and constructor arguments; typically returned by a `static Props props()` method. |
| `sys.actorOf(props, "name")` | Creates an actor inside `ActorSystem` and returns its `ActorRef`. |
| `ActorRef` | Reference to an actor incarnation. **Remains valid throughout the actor lifecycle (even if it stops and restarts)**. Can be passed inside messages. |
| `ref.tell(msg, senderRef)` | Sends `msg` asynchronously. Pass `self()` when sending from an actor, or `ActorRef.noSender()` when no reply is needed. |
| `self()` / `sender()` | When processing a message: `self()` returns this actor's `ActorRef`; `sender()` returns the sender's `ActorRef`. Reply via: **`sender().tell(msg, self());`** |

### 1.4 Actor Lifecycle & Mailbox Semantics (Slides 16, 18, 22–26)

> **Mailbox Semantics — 3 Key Rules to Remember:**
>
> 1. **Clause Matching Order:** When a message arrives, the actor checks all `match` clauses **in the exact order in which they are defined** and invokes the method of the **first matching clause**. If no matching clause exists, **the message is silently discarded**.
> 2. **FIFO Ordering Policy:** Default mailbox policy is **FIFO**. Messages from a **single sender** are processed in **FIFO order**; messages from **different senders** interleave **non-deterministically**.
> 3. **Immutable Messages:** Messages **must be immutable** ( `final` fields). This cannot be statically checked by the framework, but mutable messages cause race conditions and unexpected behaviors!

- **Lifecycle Hooks (overridable in `AbstractActor`):**
  - `preStart()` : called on the new instance right after `actorOf(...)` .
  - `preRestart(reason, msg)` : called on the **old** instance before a restart.
  - `postRestart(reason)` : called on the **new** instance that replaces the old one.
  - `postStop()` : called when stopped ( `stop()` , `context.stop()` , or `PoisonPill` ); path can be reused and `Terminated` is sent to watchers.
- **Dynamic Behavior Change ( `getContext().become()` ):**
  - Takes a `Receive` object with a new set of match clauses and replaces the actor's active receiving behavior at runtime.

```java
// Behavior switching (AlarmActor, Slide 25)
public class AlarmActor extends AbstractActor {
    @Override
    public Receive createReceive() { return disabled(); }

    private final Receive enabled() {
        return receiveBuilder()
            .match(TriggerMsg.class, this::onTrigger)
            .match(DeactivateMsg.class,
this::onDeactivate)
            .build();
    }
    private final Receive disabled() {
        return receiveBuilder()
            .match(ActivateMsg.class, this::onActivate)
            .build();
    }
    private void onActivate(ActivateMsg m) {
        getContext().become(enabled());
    }
    private void onDeactivate(DeactivateMsg m) {
        getContext().become(disabled());
    }
}
```

## 2. Lecture 2 — Communication, Stash, Ask Pattern & Distribution

### 2.1 Best-Effort Communication & Addressing (Slides 4–8)

- **No Mediating Entity:** Unlike channel-based models, communication is direct (unmediated).
- **Best-Effort Guarantees ("Less is more"):**
  - **At-most-once delivery:** no duplicates, but **messages can be lost**.
  - **No global ordering guarantees:** in Akka, FIFO order between any sender–receiver pair holds only when using TCP (or local memory).
  - **Rationale:** IP is best-effort and TCP cannot mask network partitions. Stronger guarantees are expensive and not always needed; developers can implement them on top of the basic actor model when required.
- **Addresses & Location Transparency:**
  - Each actor is identified by an **address** that abstracts away physical location (same machine, same cluster, or remote site).
  - **1 address → multiple actors:** used for load balancing, proxying, routing.
  - **1 actor → multiple addresses:** an actor can have more than one address.

### 2.2 Stash ( `AbstractActorWithStash` — Slide 9)

- Allows an actor to **postpone a message** that cannot be processed in its current state/behavior for later processing.
- Requires inheriting from `AbstractActorWithStash` :
  - `stash()` : saves the currently processed message into the actor's stash.
  - `unstashAll()` : extracts all messages from the stash **in the same FIFO order in which they were added** and prepends them to the front of the mailbox.

### 2.3 Waiting for a Reply: Workaround vs. Ask Pattern (Slides 10–11)

Because `tell()` is asynchronous, the sender does not wait for a reply. Two solutions exist when sequential request-reply is needed:

| Feature | 1. Non-Blocking Workaround ( `become + stash` ) | 2. Blocking Ask Pattern ( `Patterns.ask + Await.result` ) |
|---|---|---|
| **Superclass** | `AbstractActorWithStash` | `AbstractActor` |
| **How it works** | 1. Send request with `server.tell(msg, self())` .<br>2. Call `getContext().become(waitingForReply())` to match only the expected reply and `stash()` all other messages.<br>3. When the reply arrives: process it, call `unstashAll()` , and revert to normal behavior ( `getContext().become(idle())` ). | 1. Call `Future<Object> future = Patterns.ask(receiver, msg, timeout)` .<br>2. The receiver replies as usual: `sender().tell(reply, self())` .<br>3. The sender blocks on the future: `Await.result(future, timeout)` . |
| **Pros / Cons** | **Non-blocking:** never ties up a dispatcher thread while waiting. | **Simpler code, but blocking:** holds a dispatcher thread until reply or timeout. |

### 2.4 Remote Distribution over TCP (Slides 13–17)

Actors can be transparently distributed across multiple machines (including propagating `ActorRef` s pointing to any remote machine):

```
// 1. Configuration file ("conf")
akka {
  actor {
    provider = remote
  }
  remote {
    enabled-transports = ["akka.remote.netty.tcp"]
    netty.tcp {
      hostname = "127.0.0.1"
      port = 6123
    }
  }
}
```

```
// 2. Server Main
Config conf = ConfigFactory.parseFile(new File("conf"));
ActorSystem sys = ActorSystem.create("Server", conf);
sys.actorOf(ServerActor.props(), "serverActor");

// 3. Client Lookup (inside ClientActor)
String serverAddr =
    "akka.tcp://Server@127.0.0.1:6123/user/serverActor";
ActorSelection server =
    getContext().actorSelection(serverAddr);
// ...and everything else remains the SAME!
```

## 3. Lecture 3 — Fault Tolerance, Supervision & Clustering

### 3.1 Supervision Trees & Fault Tolerance (Slides 4–13)

- **Supervision Hierarchy:** Every actor is supervised by its parent—the actor that created it via `getContext().actorOf(...)` (and can stop it via `getContext().stop(...)`). If a supervisor cannot handle a fault locally, it escalates it up the tree.
- **Design Principle:** If an actor holds **important data that must not be lost**, it acts as a **supervisor** and delegates any dangerous/fault-prone subtask to child actors. Normally, a supervisor supervises children of the same type/functionality; if more flexibility is needed, build a **deeper hierarchy**.
- **Restart Semantics:** When a supervised actor is restarted:
  - Its **address ( `ActorRef` ) does not change**.
  - Its **mailbox lives outside of the actor memory** and is preserved across restarts.
- **Supervision Strategies & Actions:**
  - One-for-one applies the directive only to the failed child; One-for-all applies it to the failed child and all siblings.
  - **Actions:** `stop()` (terminate permanently), `restart()` (new instance, reset state), `resume()` (keep instance & state), `escalate()` (propagate to parent).

```
// Supervisor Actor (Slide 11)
public class CounterSupervisorActor extends AbstractActor
{
    private static SupervisorStrategy strategy =
        new OneForOneStrategy(
            10, // max 10 faults
            Duration.ofMinutes(1), // within 1 minute
            DeciderBuilder.match(Exception.class,
                e -> SupervisorStrategy.restart())
            .build());

    @Override
    public SupervisorStrategy supervisorStrategy() {
        return strategy;
    }
    // Spawns child in its context when receiving Props
}
```

```
// Main spawning Child via Supervisor (Slide 13)
Duration timeout = Duration.create(5, SECONDS);
ActorSystem sys = ActorSystem.create("System");
ActorRef supervisor = sys.actorOf(
    CounterSupervisorActor.props(), "supervisor");

Future<Object> waitingForCounter = ask(supervisor,
    Props.create(CounterActor.class), 5000);
ActorRef counter =
    (ActorRef) waitingForCounter.result(timeout, null);

counter.tell(new DataMessage(NORMAL_OP), noSender());
counter.tell(new DataMessage(FAULT_OP), noSender());
counter.tell(new DataMessage(NORMAL_OP), noSender());
```

### 3.2 Akka Clustering (Slides 14–20)

| Term / Mechanism | Key Rules & Exam Takeaways |
|---|---|
| **Node** | Logical member of a cluster identified by `hostname:port:uid` (an actor system/process). Multiple nodes can run on the same physical machine. |
| **Cluster & Membership** | Set of nodes joined through a **decentralized, peer-to-peer membership service** (no single point of failure/bottleneck). Cluster membership and the actors running on a node are **decoupled**. |
| **Gossip & Vector Clocks** | Nodes form an overlay network and propagate their membership view via a **gossip protocol**. Messages contain **vector clocks** to record a partial order of events (joining, leaving) so state eventually converges. |
| **Gossip Convergence** | Converges at a node when it can prove its observed cluster state is seen the same by all other nodes. **Convergence CANNOT occur while any node is `"unreachable"`** (it must become reachable again or be removed). |
| **Leader & Seed Nodes** | **Leader:** single node role managing cluster convergence. **Seed nodes:** contact points for joining; a new node contacts **all seed nodes** (at least one must be active) and sends a **join command to the first seed node that answers**. |
| **Cluster Tools** | 1. **Cluster singleton:** ensures a single actor of a given type exists in the cluster.<br>2. **Cluster sharding:** distributes actors across nodes while communicating without knowing physical location.<br>3. **Distributed data:** distributed key-value store across cluster nodes.<br>4. **Cluster metrics:** publishes/collects health metrics about cluster members. |

## 4. Lab 1 — Complete Exercise & Code Reference

| Exercise / Package | Problem Statement | Implementation & Key Details |
|---|---|---|
| **Base Counter** `akka.base` | 16 threads send 1000 `SimpleMessage` s in parallel. | `CounterActor` increments private `counter` in `onMessage` . Because message processing is atomic and serialized per actor, the final count is always `1000` with zero locks. |
| **Ex 1A — Classes** `akka.ex1.classes` | Support both increment and decrement using separate message classes. | `IncrementMsg` and `DecrementMsg` . `createReceive()` defines two clauses: `.match(IncrementMsg.class, this::onIncrement)` and `.match(DecrementMsg.class, this::onDecrement)` . Strong compile-time type safety; each message carries only its own fields. |
| **Ex 1B — Selectors** `akka.ex1.selectors` | Support increment and decrement using content predicates. | Single `CounterMsg` with `enum Op { INCREMENT, DECREMENT }` . Uses `.match(CounterMsg.class, msg -> msg.getOp() == CounterMsg.Op.INCREMENT, this::onIncrement)` . Useful when routing depends on dynamic payload values/op-codes. |
| **Ex 2 — Stash** `akka.ex2` | Keep `counter >= 0` on sequence `D1, D2, I1, D3, I2, I3` . | `CounterActor extends AbstractActorWithStash` . In `onDecrement` : if `counter > 0` then `--counter` , else `stash()` . In `onIncrement` : `++counter` then `unstashAll()` . **Trace:** `D1` stashed → `D2` stashed → `I1` (**1**, unstash `D1,D2` ) → `D1` (**0**) → `D2` stashed → `D3` stashed → `I2` (**1**, unstash `D2,D3` ) → `D2` (**0**) → `D3` stashed → `I3` (**1**, unstash `D3` ) → `D3` (**0**). Evolution: **`1, 0, 1, 0, 1, 0`** . |
| **Ex 3 — Contact List** `akka.ex3` | Client/Server contact directory ( `PutMsg` , `GetMsg` , `ReplyMsg` ). | **`ServerActor` :** stores contacts in a plain `HashMap<String, String>` ; on `GetMsg` , replies via `sender().tell(reply, self())` . **`ClientActor` (default):** extends `AbstractActorWithStash` . Starts in `idle()` . On `GetMsg` , sends `server.tell(msg, self())` and calls `getContext().become(waitingForReply())` , where `ReplyMsg` is handled and `.matchAny(msg -> stash())` postpones other commands. On `ReplyMsg` , prints result, calls `unstashAll()` , and calls `getContext().become(idle())` . **`ClientAskActor` ( `"ask"` arg):** calls `Patterns.ask(server, msg, TIMEOUT_MS)` and blocks on `Await.result(future, timeout)` . |